FREE LIVE SESSION · 30 MINUTES

Python for AI-ML

Take a real data file, teach a computer to find fraud in it, and get an AI to explain the result.

PRESENTER Dr. Emmanuel Naweji **LIVE** Mondays & Thursdays · 7:30 PM Central · 30 min

TONIGHT'S 30 MINUTES

Welcome! Here's the plan

0–2 min

Welcome

Say hi in the chat: your name, where you're joining from, and your role.

2–20 min

Three modules

One real idea from each module of Python for AI-ML, with the lab behind it.

20–23 min

Keep going

How to get the full course, every lab, and support along the way.

23–30 min

Your questions

Live Q&A. Drop questions in the chat any time; I'll answer at the end.

WHY THIS COURSE

Raw CSV to trusted model to plain-English summary, cost written down

01

Make it run anywhere

Most Python failures are not code bugs. They are setup problems. Fix the setup once.

02

Make it honest

A model learns from examples. Trust comes from how you test it, not from accuracy.

03

Make it explain itself

Ask an AI to explain the result from Python, and know what each request costs.

THE COURSE MAP

Three modules, three phrases

MODULE 1

Python That Runs Anywhere

Box it. Pin it. Rebuild it. Inspect a real dataset with plain Python.

MODULE 2

A Model You Can Trust

Don't trust the Desert Forecaster. Train two fraud models and score them fairly.

MODULE 3

Make It Explain Itself

Check the meter. Get a plain-English summary from an AI, with its cost.

01

MODULE 1 OF 3

Python That Runs Anywhere

Most Python failures are not code bugs. They are setup problems.

It ran on Friday. It broke on Monday.

Fri 5pm

the code works

pip

a teammate updates one tool everyone shares

Mon 9am

same code, new laptop, new Python

3 hours

lost before anyone finds the cause

The code was fine. Nobody wrote down which tools it needed, so nobody could rebuild it.

Box it. Pin it. Rebuild it.

1

Box it

python3 -m venv .venv: a private copy of Python inside the project folder.

2

Pin it

pip freeze > requirements.txt: every tool and its exact version, written down.

3

Rebuild it

pip install -r requirements.txt recreates the same box on any machine.

The 20% of Python: four shapes hold AI data

01

A list

Items in order, counted from 0. scores = [0.2, 0.9, 0.4]; scores[0] is 0.2.

02

A dictionary

Labels with values: one record. txn["amount"] gives 42.5.

03

A list of dicts

A table: one dict per row, like {"amount": 910.0, "is_fraud": 1}.

04

A function

A named step you can reuse, like fraud_rate(rows).

Real files are messy: blank cells, numbers stored as text, missing files. Check for them, and fail loudly.

The commands you'll type in every project

TERMINAL

```
$ python3 -m venv .venv
$ source .venv/bin/activate
$ pip install -r requirements.txt
$ pip freeze | grep -i pandas
```

`-m venv .venv`
make the box inside the project folder

`activate`
step inside it. Windows: .venv\Scripts\Activate.ps1

`install -r`
install every tool on the list, exact versions

`pip freeze`
list what's installed; grep filters to pandas

YOU SHOULD SEE your prompt start with (.venv), then pandas==3.0.5. Same version as everyone else.

MODULE 1 · RUNS ANYWHERE

Lab: inspect a real dataset, plain Python

5,000

card transactions loaded

83

blank amounts we must deal with

137

rows are fraud

2.74%

fraud rate: a rare event

A model that always says 'not fraud' scores 97.3% accuracy. When something is rare, accuracy can fool you.



Delete your box. Then rebuild it.

Same 5,000 rows and 2.74%? You just rebuilt a project from its recipe. A stranger can now run your code.

02

MODULE 2 OF 3

A Model You Can Trust

A model learns from examples. Trust comes from how you test it.



Don't trust the Desert Forecaster.

In the desert, 'no rain' is right almost every day, and useless the one day you need an umbrella.

Clean before you train: three decisions

01

Blank cells

Drop or fill. Dropping 83 of 5,000 is safe; dropping 40% is not.

02

Numbers as numbers

Text that looks like a number isn't one. pd.to_numeric fixes it.

03

Words into numbers

Models only read numbers. One-hot turns 'fuel' into a 1/0 column.

pandas does Module 1's 40 lines in about 4. Write each decision down: next month's data needs the same three.

Every model, the same three verbs

01

Fit = study

The model learns from examples with answers. X is the clues; y is the answer.

02

Predict = the exam

It guesses answers for examples it has never seen: the hidden test set.

03

Score = the grade

Compare its guesses with the real answers. Pick the score that tells the truth.

Split before you look: hide 20% as the quiz. random_state=42 and stratify=y keep the quiz fair and repeatable.

Same test set. Two stories.

ALWAYS 'NOT FRAUD'

97.3% accuracy · F1 = 0.00

984 test rows, 27 of them fraud. It catches 0. Right most of the time, useless when it matters.

VS

RANDOM FOREST

92.1% accuracy · F1 = 0.38

Lower accuracy, but the only one catching real fraud: 24 of 27, with 75 false alarms.

Precision: when it says fraud, is it right? Recall: how much fraud did it catch? F1 needs both, and can't be gamed.

Lab: train both models in one script

TERMINAL

```
$ python3 modules/module-2/scripts/train.py
$ cat metrics.json
```

train.py
load, clean, split, train two models, score, save

always_not_fraud
accuracy 0.973 · f1 0.000

logistic_regression
accuracy 0.887 · f1 0.302

random_forest
accuracy 0.921 · f1 0.381: the winner

YOU SHOULD SEE 3933 train rows · 984 test rows, three result lines, and saved -> metrics.json. Trust the F1 column.

Lab: train a fraud model you can trust

1

Look before you train

groupby shows electronics (4.6%) and online (3.2%) have the most fraud.

2

Train both models

train.py cleans, splits 80/20, fits two models and saves metrics.json.

3

Find the three verbs

m.fit, m.predict, f1_score: the same three lines for both models.

4

Write down the target

0.381 is the score to beat. Every change you try is compared to it.

03

MODULE 3 OF 3

Make It Explain Itself

Asking an AI is a request with a key, a message and a price. Write it down.

One call costs nothing. Ten thousand don't.

88

tokens we send (input)

77

tokens we get back (output)

$0.0014

one call, at $3 in / $15 out per million

≈ $426

a month at 10,000 calls a day

Output tokens cost 5× input here. max_tokens limits the expensive part.

An LLM is an API: three in, two back

IN

model · max_tokens · messages

Which AI to use, the longest reply allowed, and your message. Bigger models cost more.

VS

BACK

content + usage

msg.content[0].text is the reply. msg.usage is the bill: how many tokens were used.

Like a function: things go in, a result comes out. It just runs on someone else's computer, and they charge by the token.

Check the meter

01

Keys stay secret

ANTHROPIC_API_KEY lives in a hidden setting, never in code. Leaked? Replace it.

02

Tokens are the bill

Words in and words out, priced separately. Read the usage line on every call.

03

Networks fail

Wait 2, 4, then 8 seconds and stop after three tries. Every retry is charged.

A taxi meter ticks for every mile. An AI meter ticks for every token. Your API key is the card that pays the fare.

Lab: run it offline, no key, no cost

TERMINAL

```
$ python3 modules/module-3/scripts/explain.py --offline
```

`explain.py`
reads Module 2's metrics.json, asks the AI to explain

`--offline`
a fake reply and estimated tokens: free

`usage line`
the meter: input tokens, output tokens, cost

`summary.md`
the plain-English explanation, saved

YOU SHOULD SEE usage: input_tokens=88 output_tokens=77 · cost $0.00142 (offline estimate), then saved -> summary.md.

Lab: explain your model, end to end

1

Read the prompt

Who it's for, the data, 4 short sentences, no jargon. You pay for every word.

2

Make one real call

Optional: set ANTHROPIC_API_KEY and drop --offline. Fractions of a cent.

3

Reprice it

A cheaper model's rates: same 88 and 77 tokens, $0.00047. About a third.

4

Run the pipeline

inspect_data.py && train.py && explain.py: raw CSV to plain English.

THE WHOLE COURSE

Three modules. Three phrases to keep.

Box it. Pin it. Rebuild it.

Module 1. venv, freeze, install -r: a stranger can rebuild and run your project.

Don't trust the Desert Forecaster

Module 2. On rare events, 97% accuracy can mean zero fraud caught. F1 can't be fooled.

Check the meter.

Module 3. Input and output tokens, priced separately, logged on every call.



You can do this.

Rebuildable, honest, explained: pick any CSV with a yes/no column and run the same three steps.

THE MEMBERSHIP

Every course. One price.

$49

per month

All 8 courses, every module and every lab, plus a live session with me every month.

ALL 8 COURSES INCLUDED

- Linux for AI-ML Systems
- Python for AI-ML
- Zero to AI-ML Systems Engineering
- DevOps for AI-ML Systems
- SRE for AI-ML Systems
- GPU Optimization for AI-ML Systems
- FinOps for AI-ML Systems
- AI Agent Operations

BUILD WITH ME, LIVE

Once a month, we build something together

1

Learn at your pace

Work through any course, any module, on your own schedule.

2

Bring a real problem

An idea, a project at work, or a lab you want to take further.

3

Meet me live

Once a month we get on a call and build it together, step by step.

4

Leave with it built

Something real you made, understand and can show.

FREE LIVE SESSIONS

Every Monday and Thursday, 7:30 PM Central

A different course each session. The full rotation takes 4 weeks, then starts again.

	WEEK 1	WEEK 2	WEEK 3	WEEK 4
MONDAY	Linux for AI-ML Systems	NEXT Zero to AI-ML Systems Engineering	SRE for AI-ML Systems	FinOps for AI-ML Systems
THURSDAY	TONIGHT Python for AI-ML	DevOps for AI-ML Systems	GPU Optimization for AI-ML Systems	AI Agent Operations

Join the membership today

$49/month · all 8 courses · a live build session with me every month

transformed2succeed.com

Next free session: Monday, 7:30 PM Central · Zero to AI-ML Systems Engineering

Questions? Let's take them now.

Dr. Emmanuel Naweji · Transformed 2 Succeed